

CSE233 Week 7: Pass-by-Reference Parameters

When we introduced functions, we only used pass-by-value parameters. Pass-by-value parameters create a copy of the argument(s)'s value when the function is called. This means that changes to the parameters within the function are not reflected outside the function. Sometimes, the function needs to make changes to its parameters. In this case, pass-by-reference can be used. Pass-by-reference passes the memory address of the argument, rather than creating a copy of its value. We will also discuss function overloading this week. Overloading allows the same name to be used for multiple functions.

Pass-by-Value Parameters

First, let's look at what happens when you call a function using pass-by-value. We'll consider a simplified view of a program's memory model. Local variables are created in an area of memory called the call stack. Each time a function is called, its variables are added to the call stack. When the function ends, all variables associated with the function are removed from the call stack. Each parameter is also created on the call stack, since parameters are special local variables that functions have. When pass-by-value is used, the values of the parameter are copied into the memory used by the parameter. Below is a table of the sizes (in bytes) of some common data types (simplified and assuming it is a 64-bit computer):

Type	Memory
int	4 bytes
float	4 bytes
long	8 bytes

Type	Memory
double	8 bytes
bool	1 byte
char	1 byte
std::string	1 byte/char

In reality, std::string uses more than one byte per character, but for large strings the extra space becomes negligible.

Now, let's look at a basic C++ program.

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}

int main() {
    int x;
    int y;
    std::cout << "Enter two integers: ";
    std::cin >> x >> y;

    int z = add(x, y);
    std::cout << x " + " << y << " = " << z << "\n";

    return 0;
}
```

When a C++ program is compiled, it looks for the function `int main()` as its entry point. This is the first function that gets called in the program. The first thing main does is create two integer variables, `x` and `y`. These are allocated on the stack:

```
| 1004 | main::y | 0 |
| 1000 | main::x | 0 |
```

Next, it prints a message to the console and reads values for `x` and `y` from the user. Let's assume that 5 and 8 were the values entered. Then, the values of `x` and `y` are updated in memory:

```
| 1004 | main::y | 8 |  
| 1000 | main::x | 5 |
```

Next, a variable `z` is created:

```
| 1008 | main::z | 0 |  
| 1004 | main::y | 8 |  
| 1000 | main::x | 5 |
```

The declaration of `z` is calling the function `add()`. Therefore, before `z` can be given a value, `add()` must run. `Add` has two parameters which are added to the call stack. Since they are pass-by-value, the arguments `x` and `y`'s values are copied into the memory for `a` and `b`.

```
| 1016 | add::b | 8 |  
| 1012 | add::a | 5 |  
| 1008 | main::z | 0 |  
| 1004 | main::y | 8 |  
| 1000 | main::x | 5 |
```

Now, `add()` immediately returns the value of adding `a` and `b`. This would occur directly on the CPU, so it does not use the RAM. The value of `5 + 8` is `13`, so that value is stored in `z`. Since `add()` is done, its variables are removed from the stack:

```
| 1008 | main::z | 13 |  
| 1004 | main::y | 8 |  
| 1000 | main::x | 5 |
```

`main()` has another print statement, and then ends. The variables for `main()` are removed from memory, and the rest of the program is also unloaded from memory now that execution has halted.

Now, let's consider another function that modifies its parameter:

```
void increment(int x) {  
    ++x;  
}  
  
int main() {  
    int x = 0;
```

```

    increment(x);
    return 0;
}

```

The function `increment()` increases its parameter's value by one. However, since it is pass-by-value, the value of `x` in `main()` will not be modified. Once again, let's look at the call stack for this program. To start off with, the variable `x` is allocated on the stack for `main()`:

```

| 1000 | main::x | 0 |

```

Then, the parameter `x` is added to the stack for `increment()`. Its value is a copy of `x`'s value.

```

| 1004 | increment::x | 0 |
| 1000 | main::x      | 0 |

```

Next, the value of `increment::x` is increased by one. Since it is a totally separate variable than `main::x`, the value of `main::x` is not changed.

```

| 1004 | increment::x | 1 |
| 1000 | main::x      | 0 |

```

Finally, `increment::x` is popped from the call stack.

```

| 1000 | main::x | 0 |

```

Now, let's look at how pass-by-reference differs.

Pass-by-Reference

Rather than passing a copy of its arguments' values, pass-by-reference passes the memory address of its arguments. When the parameters are used in the body of the function, the memory address is "dereferenced", meaning that program reads or writes to the memory address being stored by the parameter rather than using the actual value. On 64-bit computers, memory addresses are typically stored using 8-bytes. Pass-by-reference parameters are specified by putting an

ampersand (the & symbol) between the type and name of the parameter. Let's take the previous `increment()` function and make its parameter pass-by-reference:

```
void increment(int& x) {
    ++x;
}

int main() {
    int x = 0;
    increment(x);
    return 0;
}
```

Now, instead of copying the value of `x` from `main`, the address of `x` is passed. This allows the function to directly modify `x`. Like before, the `x` in `main()` will be allocated on the call stack:

	1000		main::x		0	
--	------	--	---------	--	---	--

Now, when `increment()` is called, it is passed the address of `main::x` instead of the value 0.

	1008		increment::x		1000	
	1004					
	1000		main::x		0	

Then, when the `++x;` statements runs, `x` will be dereferenced, allowing the value in `main()` to be modified:

	1008		increment::x		1000	
	1004					
	1000		main::x		1	

Since the value of `main::x` was modified directly, the change stays after `increment::x` is removed from the call stack:

	1000		main::x		1	
--	------	--	---------	--	---	--

One difference when using pass-by-reference compared to pass-by-value is that only variables can be passed by reference:

```

// this function uses pass-by-value
int doubleNumber(int x) {
    return x * 2;
}

// this function uses pass-by-reference
int tripleNumber(int& x) {
    return x * 2;
}

int main() {
    // this is legal
    int x = doubleNumber(5);
    // this is illegal, the program won't compile
    int y = tripleNumber(6);

    return 0;
}

```

As can be seen, this results in a compilation error:

```

main.cpp: In function 'int main()':
main.cpp:15:26: error: cannot bind non-const lvalue reference of type
 15 |     int y = tripleNumber(6);
    |                      ^
main.cpp:7:23: note:   initializing argument 1 of 'int tripleNumber(int&)
 7 | int tripleNumber(int& x) {
    |             ~~~~~^

```

This makes pass-by-reference parameters more restrictive than pass-by-value, since we can no longer pass literal values or expressions as arguments.

Constant References

There may be times (which are discussed in the applications section) where a variable needs to be passed-by-reference, but the function should not be allowed to modify the parameter. In that case, a constant reference can be used. By placing `const` before the parameter's type, the compiler will check that the function does not modify the parameter. If the function does modify a constant reference, the program will fail to compile. Another consequence of constant references is that literal values and expressions can be passed as arguments. Below is an example of using a constant reference:

```

#include <iostream>
#include <string>

void printLine(const std::string& msg) {
    std::cout << msg << "\n";
}

int main() {
    std::string message = "Hello, World!";
    // can pass a variable
    printLine(message);
    // with a const reference, a value
    // can be directly passed as well
    printLine("Goodbye");

    return 0;
}

```

Applications of Pass-by-Reference

Pass-by-reference is primarily used when functions need to modify their parameters, however it is also commonly used to maximize memory efficiency.

Updating Values

The most obvious use of pass-by-reference parameters is when the value of the parameter is used, and needs to be updated by the function. An example is a swap function, where two values are swapped. This doesn't work with pass-by-value, since the values of the two variables will only be updated within the function.

```

void swap(int& x, int& y) {
    // save the value of x in temp
    int temp = x;
    // copy the value of y into x
    x = y;
    // now assign temp to y (y is now x's original value)
    y = temp;
}

```

Output Parameters

Functions can only return a single value. Sometimes, you may wish to return multiple values from a function. To do so, you can use pass-by-

reference parameters. Here is an example that reads a user's first and last name using a function `readName()` that uses two pass-by-reference parameters to "return" the two values to `main()`.

```
#include <iostream>
#include <string>

void readNames(std::string& first, std::string& last) {
    std::cout << "Enter first name: ";
    std::cin >> first;
    std::cout << "Enter last name: ";
    std::cin >> last;
}

int main() {
    std::string firstName;
    std::string lastName;
    readNames(firstName, lastName);

    std::cout << "Hello, " << firstName << " " << lastName << "\n";

    return 0;
}
```

In this case, the values of the parameters are irrelevant (in fact, we don't even assign them values before calling `readNames()`). Instead of using the parameters to pass values into the function, we are using them to pass values out of the function.

Efficiently Passing Large Parameters

For basic data types like `int`, `float`, `long`, `double`, etc., there are no memory savings by passing parameters by reference. Passing a reference takes 8 bytes on most modern hardware, and primitive data types use 8 or fewer bytes. However, let's consider strings. A single character uses 1 byte. A string is an ordered collection of characters, so a string with 4 characters uses 4 bytes (plus some overhead). A string with 100 characters will use 100 bytes. While the savings for a small string may be negligible, for large strings, passing them by reference can save a significant amount of memory. In a couple of weeks, we will discuss classes and structs, which typically use more memory than a primitive data type as well. For classes, structs, and strings, it is normally better to pass them as a constant reference than to pass them by value.

There are some exceptions, but those are beyond the scope of this course. For now, any time you would pass a string or object by value, pass it as a constant reference instead.

Overloading

C++ supports function overloading. Overloading allows the same name to be used for multiple functions. Functions are differentiated by their parameter count and types. Functions cannot be overloaded by their return type or parameter names. Consider this function

`computeTotal()` :

```
double computeTotal(double subtotal, double tax) {  
    double taxAmount = subtotal * tax;  
    return taxAmount + tax;  
}
```

Now, let's overload this function so that it computes the total based on the unit price of an item, quantity of that item, and tax:

```
double computeTotal(double subtotal, double tax) {  
    double taxAmount = subtotal * tax;  
    return taxAmount + tax;  
}  
  
// this version has a third parameter, allowing it to be overloaded  
double computeTotal(double itemPrice, double tax, int quantity) {  
    double subtotal = itemPrice * quantity;  
    return computeTotal(subtotal, tax);  
}
```

Now, we can technically overload this function based on the item price and quantity (either there is no tax or the tax is baked into the item price). This is legal since `quantity` is an `int`, and tax is a `double` :

```
double computeTotal(double subtotal, double tax) {  
    double taxAmount = subtotal * tax;  
    return taxAmount + tax;  
}  
  
// this version has a third parameter, allowing it to be overloaded  
double computeTotal(double itemPrice, double tax, int quantity) {  
    double subtotal = itemPrice * quantity;  
    return computeTotal(subtotal, tax);  
}
```

```

}

// this version has two parametes, but the second parameter
// has a different data type than the first version
// this is also a legal overload
double computeTotal(double itemPrice, int quantity) {
    return itemPrice * quantity;
}

```

Here is another example of overloading. This time, we'll overload the `swap()` function from before so that it can swap other data types:

```

long swap(long& x, long& y) {
    long temp = x;
    x = y;
    y = temp;
}
void swap(double& x, double& y) {
    double temp = x;
    x = y;
    y = temp;
}
void swap(std::string& x, std::string& y) {
    std::string temp = x;
    x = y;
    y = temp;
}

```

Function overloading should be used sparingly. If two functions do the same thing, but need different parameters, then overloading works. However, if the functions perform different operations, then different names should be used.

Conclusion

This week, we discussed pass-by-reference parameters. Pass-by-reference parameters allow the changes made to parameters within the function to be reflected outside the function. We looked at the difference between how pass-by-value and pass-by-reference are represented in memory. Then, we discussed some applications of pass-by-reference. Passing a constant reference is used to allow more efficient passing of large data types like strings. Finally, we looked at function overloading. Function overloading allows the same function name to be used for multiple functions.

